

Weather AQI Predictor

Complete Detailed Project Report

End-to-end machine learning pipeline for UAE AQI forecasting with Hopsworks Feature Store, live Open-Meteo data, multi-horizon ML models, and a premium Streamlit dashboard.

Feature Store	Model Artifact	Dashboard	Forecasting
Hopsworks Feature Store	Hopsworks Resources	Streamlit Premium UI	Day 1, Day 2, Day 3, 3-Day Avg

Final implementation status: Feature Store completed, model artifact uploaded, dashboard completed, Model Registry attempted and documented.

Table of Contents

1. Project Overview

2. Final Project Outcome

3. Original Project Requirements

3.1 Feature Pipeline

3.2 Historical Backfill

3.3 Training Pipeline

3.4 Automation

3.5 Web App Dashboard

3.6 Additional Guidelines

4. Final System Architecture

5. Main Files Created and Used

5.1 src/config.py

5.2 src/feature_pipeline.py

5.3 src/hopsworks_store.py

5.4 src/training_pipeline.py

5.5 src/app.py

5.6 upload_model_artifact_hopsworks.py

6. Data Sources

7. Supported UAE Cities

8. Feature Engineering

8.1 Time Features

8.2 Pollutant Features

8.3 Weather Features

8.4 Lag Features

8.5 Rolling Features

8.6 Interaction Features

8.7 Future Weather Features

9. Target Engineering

10. Feature Store Implementation

11. Hopsworks Feature Store Problems and Fixes

12. Training Pipeline

13. Model Performance

14. Model Registry and Model Artifact Storage

15. Streamlit Dashboard

15.1 Sidebar Controls

15.2 Premium Hero Section

15.3 Main KPI Cards

15.4 Source / Status Card

15.5 Forecast Summary

15.6 AQI Scale

15.7 Health Recommendations

15.8 AQI Trend Intelligence

15.9 Major Air Pollutants

15.10 City Comparison

15.11 Why This Prediction?

15.12 Model Performance

15.13 Downloads

16. Dashboard UI Problems and Fixes

17. Timezone Error and Fix

18. API Key and Permission Problems

19. Environment Configuration

20. Commands Used

20.1 Run Feature Pipeline

20.2 Run Training Pipeline

20.3 Run Dashboard

20.4 Upload Existing Model Artifact to Hopsworks Resources

20.5 Check Python Syntax

21. Final Dashboard Source Card

22. Final Technical Stack

23. What Was Achieved

24. Problems Faced and How They Were Solved

25. Limitations

26. Future Improvements

27. Final Conclusion

1. Project Overview

The Weather AQI Predictor is an end-to-end machine learning system designed to forecast the Air Quality Index (AQI) for UAE cities over multiple future horizons. The main goal of the project was to build a working data pipeline that collects weather and pollutant data, engineers meaningful ML features, stores them in a feature store, trains forecasting models, and displays real-time plus predicted AQI in an interactive dashboard.

The original project brief required a system that could predict AQI for the next 3 days using a serverless-style pipeline. The required workflow included raw weather/pollution data ingestion, feature generation, feature store storage, model training, model registry storage, automation, and a Streamlit/Gradio/FastAPI-style web app for predictions.

2. Final Project Outcome

By the end of the project, the system achieved the following:

Component	Final Status
Historical AQI + Weather Data Pipeline	Completed
Multi-city UAE data support	Completed
Feature engineering pipeline	Completed
Hopsworks Feature Store upload	Completed
Feature Group	aqi_features_clean v2
Local feature store backup	Completed
Multi-horizon ML training	Completed
Model evaluation using RMSE, MAE, R ²	Completed
Model artifact local save	Completed
Model artifact uploaded to Hopsworks Resources	Completed
Streamlit dashboard	Completed
Live AQI and live forecast weather integration	Completed
Pollutant breakdown UI	Completed
AQI trend intelligence chart	Completed
City comparison	Completed
Health recommendations	Completed
Model explainability / feature drivers	Completed
Hopsworks Model Registry	Attempted, but blocked by Hopsworks system dataset issue

The final system is not just a static ML notebook. It is a full pipeline-based application with ingestion, storage, training, prediction, visualization, and Hopsworks cloud integration.

3. Original Project Requirements

The original brief expected the following major parts:

3.1 Feature Pipeline

The project needed a Python script that could fetch weather and pollutant data from an external API, compute model input features and output targets, include time-based features such as hour/day/month, and store those features in a feature store such as Hopsworks or Vertex AI.

3.2 Historical Backfill

The feature script needed to be run for a range of past dates so that enough historical training data could be generated for the ML models.

3.3 Training Pipeline

The training script needed to fetch historical features and targets from the Feature Store, train and evaluate models, test different model types, evaluate using RMSE, MAE, and R^2 , and store the trained model in a Model Registry.

3.4 Automation

The brief also suggested automation through tools such as GitHub Actions or Apache Airflow, with the feature script running hourly and the training script running daily.

3.5 Web App Dashboard

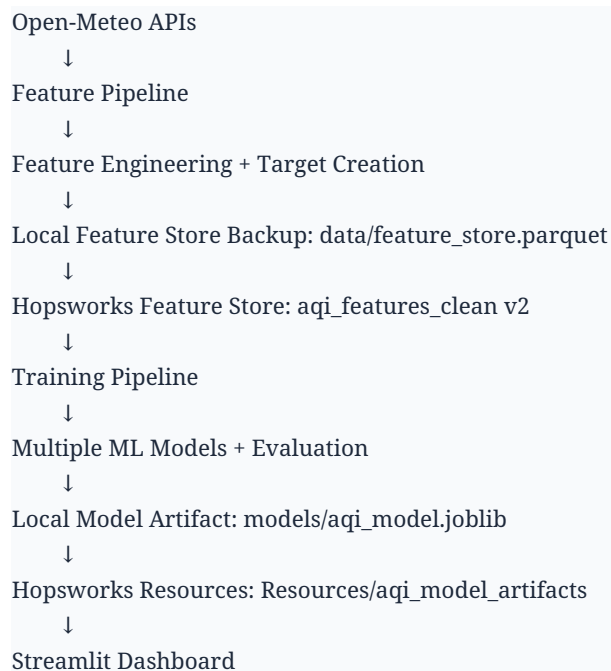
The app needed to load the model and features from the Feature Store, compute predictions, and display them through a descriptive dashboard using tools such as Streamlit, Gradio, Flask, or FastAPI.

3.6 Additional Guidelines

The project also recommended EDA, multiple forecasting models, SHAP/LIME-style explainability, hazardous AQI alerts, and a final detailed report.

4. Final System Architecture

The final architecture follows this flow:



The architecture was designed with fallback support. If Hopsworks is unavailable, the system can still run from local Parquet and local model backups. This made the project more stable during development and testing.

5. Main Files Created and Used

5.1 src/config.py

This file stores central configuration values such as:

Feature store path

Model path

Metrics path

Required ML feature columns

Environment-driven configuration

This made the project easier to maintain because feature pipeline, training pipeline, and dashboard could all use shared paths and constants.

5.2 src/feature_pipeline.py

This is the main feature generation pipeline. It handles:

Fetching historical AQI and weather data

Fetching pollutant data

Creating city-level records

Computing lag features

Computing rolling mean and rolling standard deviation features

Creating target columns for future AQI prediction

Saving local Parquet backup

Uploading data to Hopworks Feature Store

5.3 src/hopworks_store.py

This file manages all Hopworks-related logic:

Connecting to Hopworks

Reading environment variables

Creating or getting feature groups

Preparing data for Hopworks schema validation

Uploading feature data in chunks

Reading feature data back from Hopworks

Attempting model registry operations

Falling back safely when Hopworks is unavailable

5.4 src/training_pipeline.py

This file trains the ML models. It handles:

Loading training data from Hopworks Feature Store

Falling back to local Parquet if Hopsworks read fails

Cleaning and preparing training data

Training multiple models

Evaluating models using RMSE, MAE, and R^2

Selecting best models per target

Saving the model bundle locally

Saving metrics JSON

Attempting Model Registry upload

5.5 src/app.py

This is the Streamlit dashboard. It handles:

Loading feature data

Loading model bundle

Selecting UAE city

Selecting forecast horizon

Fetching live AQI

Fetching live weather

Fetching future weather forecast

Updating inference features using live forecast weather

Generating AQI predictions

Showing current and predicted AQI

Showing pollutant breakdown

Showing city comparison

Showing AQI trend intelligence

Showing model explanation

Showing technical source information

5.6 upload_model_artifact_hopsworks.py

This file was created after Hopsworks Model Registry caused a system dataset issue. It uploads the local trained model artifact and metrics to Hopsworks Resources:

Resources/aqi_model_artifacts/aqi_model.joblib

Resources/aqi_model_artifacts/metrics.json

Resources/aqi_model_artifacts/README.md

This became the final practical solution for storing the model artifact in Hopsworks.

6. Data Sources

The project used Open-Meteo APIs for AQI, pollutant, weather, and forecast weather data.

The dashboard uses:

Live AQI source: Open-Meteo current AQI

Future weather source: Open-Meteo Forecast API

Historical feature source: Hopsworks Feature Store or local Parquet backup

Pollutants included in the dashboard and pipeline:

PM2.5

PM10

Carbon Monoxide

Nitrogen Dioxide

Sulphur Dioxide

Ozone

Dust

The AQI standard used in the dashboard is AQI-US.

7. Supported UAE Cities

The project was developed for multiple UAE cities:

City	Dashboard Emoji / Identity
Abu Dhabi	
Ajman	
Al Ain	
Dubai	
Ras Al Khaimah	
Sharjah	

The dashboard initially used plain city names, but later city-specific icons/emojis were added to make the interface more premium and visually understandable.

8. Feature Engineering

Feature engineering was one of the most important parts of the project because AQI prediction depends heavily on pollutant trends, weather movement, and time-based patterns.

The pipeline generated features such as:

8.1 Time Features

Hour

Day

Month

Weekday

Weekend indicator

Cyclical time encodings where applicable

These helped the model understand daily and seasonal AQI patterns.

8.2 Pollutant Features

Core pollutant columns included:

pm2_5

pm10

carbon_monoxide

nitrogen_dioxide

sulphur_dioxide

ozone

dust

8.3 Weather Features

Weather-related columns included:

Temperature

Relative humidity

Pressure

Wind speed

Wind direction

Precipitation

These are important because pollution concentration changes with wind, humidity, temperature, and pressure.

8.4 Lag Features

Lag features were created to capture recent AQI and pollutant history, such as:

AQI lag values

Pollutant lag values

Dust lag values

PM2.5 and PM10 lag values

These help the model learn whether AQI is rising, falling, or stable.

8.5 Rolling Features

Rolling averages and rolling standard deviations were created for different windows, such as:

12-hour rolling mean

24-hour rolling mean

48-hour rolling mean

Rolling standard deviation

These help smooth short-term noise and expose trend direction.

8.6 Interaction Features

Interaction features were also added, such as:

PM2.5 to PM10 ratio

AQI to PM ratio

Wind and pollutant interactions

Humidity and pollutant interactions

Temperature and humidity interaction

These improved model understanding of combined environmental effects.

8.7 Future Weather Features

A major improvement was adding live forecast weather inputs. Instead of using only old saved weather features, the app now fetches future weather and uses it during inference.

This made the dashboard more realistic because AQI prediction for the next day should consider upcoming temperature, wind, humidity, and rain conditions.

9. Target Engineering

The project was upgraded from a single 72-hour average forecast to a multi-horizon AQI forecasting system.

Final targets included:

Target Key	Target Column	Meaning
day_1	target_aqi_day1_avg	Average AQI for next 24 hours
day_2	target_aqi_day2_avg	Average AQI for next 24–48 hours
day_3	target_aqi_day3_avg	Average AQI for next 48–72 hours
next_72h_avg	target_aqi_next_72h_avg	Average AQI for next 72 hours

This made the dashboard more useful because users can choose:

Day 1 Forecast

Day 2 Forecast

Day 3 Forecast

3-Day Average

10. Feature Store Implementation

The local feature store was saved at:

data/feature_store.parquet

The cloud feature store was implemented using Hopsworks.

Final Hopsworks configuration:

HOPSWORKS_PROJECT_NAME=uae_aqi_predictor
USE_HOPSWORKS=true
FEATURE_GROUP_NAME=aqi_features_clean
FEATURE_GROUP_VERSION=2

```
HOPSWORKS_UPLOAD_CHUNK_SIZE=3500
```

```
HOPSWORKS_UPLOAD_RETRY_COUNT=3
```

The final successful Hopsworks feature group was:

```
aqi_features_clean v2
```

The final upload successfully completed:

```
Chunk 31/31 uploaded successfully.
```

Hopsworks upload successful.

Cloud feature store status: Hopsworks upload successful.

This confirmed that the feature data was successfully stored in Hopsworks.

11. Hopsworks Feature Store Problems and Fixes

Problem 1: Old Feature Group Schema Validation Failed

Initially, the feature group had a different schema. New engineered features such as rolling, ratio, and interaction columns did not exist in the old Hopsworks feature group.

The error was similar to:

```
column does not exist in feature group
```

Solution

A new clean feature group was created:

```
aqi_features_clean v2
```

This avoided conflicts with the old schema.

Problem 2: location_key Primary Key Null Values

Hopsworks rejected the upload because the primary key column contained null values.

Error:

```
Primary key column location_key contains null values
```

Solution

The feature pipeline was updated to ensure:

```
location_key was always generated
```

```
location_key was converted to string
```

```
rows with invalid keys were cleaned before upload
```

Problem 3: Hopsworks Upload Timed Out

Uploading 105,552 rows at once caused Kafka timeout errors:

```
KafkaError: Local: Message timed out
```

Solution

```
Chunk upload was implemented.
```

The final upload used:

31 chunks of up to 3500 rows

This solved the timeout issue and allowed Hopsworks upload to complete successfully.

Problem 4: Hopsworks .insert() Returned None

At one stage, Hopsworks returned:

Hopsworks returned None for feature group object

Solution

The Hopsworks helper code was improved to:

check whether feature group exists

create the feature group if missing

safely handle None

print clearer status messages

fall back to local Parquet if cloud upload fails

12. Training Pipeline

The training pipeline loads data from Hopsworks Feature Store when available. If Hopsworks is unavailable, it falls back to the local Parquet file.

Training data reached approximately:

105,552 rows

277+ columns

238 usable ML features

The training pipeline trained models for each target:

Day 1

Day 2

Day 3

3-Day Average

Models tested included:

Ridge

Elastic Net

Random Forest

Gradient Boosting

Hist Gradient Boosting

Extra Trees

XGBoost where available

Evaluation metrics:

RMSE

MAE

R^2

The best model varied by target. In one final run, the Day 1 model achieved strong performance, with R^2 around the high 0.8 range, while longer horizon forecasts were naturally harder and had lower accuracy.

13. Model Performance

The dashboard shows model performance in an understandable way instead of only showing raw technical metrics.

It explains:

Best model

Accuracy level

Expected error range

Forecast target

RMSE

MAE

R^2

This helped convert technical ML metrics into user-friendly dashboard information.

Example explanation:

RMSE: The model prediction is usually around X AQI away from the actual AQI.

MAE: On average, the model misses the real AQI by about X AQI.

R^2 : The model explains about X% of the AQI pattern.

14. Model Registry and Model Artifact Storage

The original requirement expected storing the trained model in a Model Registry. The training code attempted to save the model to Hopworks Model Registry, but Hopworks returned this error:

No Models dataset exists in project uae_aqi_predictor.

Please enable the Serving service or create the dataset manually.

We tried several solutions:

Opened Model Registry page in Hopworks.

Checked Deployments page.

Tried to create a deployment.

Tried to manually create a Models dataset.

Hopworks rejected Models dataset creation because it is a reserved/system dataset.

Created a separate upload script to store the model artifact in Hopworks Resources.

Final practical solution:

```
Resources/aqi_model_artifacts/aqi_model.joblib
Resources/aqi_model_artifacts/metrics.json
Resources/aqi_model_artifacts/README.md
```

The upload completed successfully.

Final model storage status:

Item	Status
Local model artifact	Saved
Local metrics JSON	Saved
Hopsworks Model Registry	Blocked by system dataset issue
Hopsworks Resources model artifact upload	Completed
Hopsworks Resources metrics upload	Completed

This means the model is still stored in Hopsworks, but under the Resources filesystem instead of the formal Model Registry UI.

15. Streamlit Dashboard

The dashboard was built using Streamlit and styled with custom HTML/CSS for a premium AQI.in-inspired yellow theme.

Final dashboard sections include:

15.1 Sidebar Controls

The sidebar allows the user to select:

City

Forecast day

Cities are shown with emojis/icons for better UX.

15.2 Premium Hero Section

The hero section shows:

Selected city

Live badge

City vibe label

Dashboard title

Short explanation of the forecast

Example:

Dubai Air Quality Index Forecast

Real-time AQI monitoring with machine learning based day 1 forecast prediction, pollutant analytics, city comparison, and premium dashboard experience.

15.3 Main KPI Cards

The top section includes three major cards:

Live AQI

Forecast AQI

Weather Inputs

The Forecast AQI card was later improved because the category text was too large and the predicted AQI number was not prominent enough. The UI was updated so the predicted value becomes the main focus.

15.4 Source / Status Card

The dashboard source card includes:

Feature Store source

Feature group name and version

Model artifact location

Model file name

Metrics file name

Live AQI source

Future weather source

Historical feature timestamp

This was added so the dashboard clearly proves that Hopsworks integration exists.

15.5 Forecast Summary

The dashboard shows:

Day 1 Forecast

Day 2 Forecast

Day 3 Forecast

3-Day Average

This helps users compare different future horizons.

15.6 AQI Scale

An AQI scale was added to visually show where the current and predicted values fall.

15.7 Health Recommendations

Recommendations are generated based on the predicted category. For example:

Sensitive groups should reduce prolonged outdoor activity.

People with asthma should carry medication.

Consider wearing a mask outdoors.

15.8 AQI Trend Intelligence

A premium AQI trend chart was added. It shows:

Historical AQI trend

Live AQI marker

Forecast AQI marker

Threshold reference line

Yellow theme background

This section was improved multiple times to match the rest of the premium theme.

15.9 Major Air Pollutants

The pollutant section shows cards for:

PM2.5

PM10

CO

NO₂

SO₂

O₃

Dust

This section was redesigned to look closer to AQI.in-style pollutant cards.

15.10 City Comparison

The dashboard compares AQI across UAE cities and highlights:

Best current air quality

Highest current AQI

Category color

15.11 Why This Prediction?

A user-friendly explanation section was added. It explains why the model predicted a specific AQI level, based on:

Current AQI

Fine particles

Recent trend

Weather movement

This section makes the ML result easier to understand for non-technical users.

15.12 Model Performance

The model performance section explains model reliability using simple language and technical metrics.

15.13 Downloads

The dashboard allows downloading:

Latest prediction CSV

Model metrics JSON

16. Dashboard UI Problems and Fixes

Problem 1: Cards Were Cut From the Top

Some dashboard cards appeared cut off because of spacing and container layout issues.

Solution

The layout spacing was adjusted, and the UI was restructured to keep cards fully visible.

Problem 2: Raw HTML Appeared on the Dashboard

At one point, pollutant cards displayed raw `<div>` HTML instead of rendering properly.

Cause

Streamlit sometimes treats indented HTML strings as code blocks if they are passed incorrectly.

Solution

The HTML rendering logic was changed to compact string-based `st.markdown(..., unsafe_allow_html=True)` blocks.

Problem 3: UI Theme Became Inconsistent

Some updates accidentally reverted the dashboard to an old plain white theme.

Solution

The yellow premium theme was restored, keeping:

Yellow gradient hero

Cream cards

Soft shadows

Rounded corners

AQI.in-inspired look

Premium spacing and typography

Problem 4: Trend Chart Looked Like Old Theme

The trend chart initially looked plain and did not match the new dashboard design.

Solution

The trend section was redesigned into a premium “AQI Trend Intelligence” card with:

Yellow background

Better line styling

Live AQI marker

Forecast marker

Legend

Improved title and description

Problem 5: Forecast AQI Card Had Wrong Visual Hierarchy

The predicted AQI value was too small, while the category text was too prominent.

Solution

The forecast card was redesigned so the predicted AQI number becomes the main focus, and the category appears as a smaller badge.

17. Timezone Error and Fix

After Hopsworks data was uploaded, the dashboard crashed with:

```
TypeError: Cannot subtract tz-naive and tz-aware datetime-like objects
```

Cause

Hopsworks returned timestamps with timezone information, but `pd.Timestamp.now()` was timezone-naive.

Solution

The timestamp freshness check was replaced with a safe timezone-aware function/block:

```
latest_time_for_check = pd.to_datetime(latest_time, errors="coerce")
if pd.notna(latest_time_for_check):
    if getattr(latest_time_for_check, "tzinfo", None) is not None:
        current_time_for_check = pd.Timestamp.now(tz=latest_time_for_check.tzinfo)
    else:
        current_time_for_check = pd.Timestamp.now()
    if current_time_for_check - latest_time_for_check > pd.Timedelta(days=3):
        st.warning(...)
```

This prevented the dashboard from crashing due to timezone mismatch.

18. API Key and Permission Problems

Problem

When uploading model artifacts to Hopsworks Resources, the API key did not have the required permission.

Error:

```
No valid scope found for this invocation.
```

Valid scope is: [DATASET_CREATE]

Solution

A new Hopsworks API key was created with dataset-related permissions such as:

Dataset create

Dataset view

Feature Store

Project access

Model-related permissions where available

After updating .env, the model artifact upload succeeded.

19. Environment Configuration

Final .env structure:

```
HOPSWORKS_API_KEY=your_api_key
HOPSWORKS_PROJECT_NAME=uae_aqi_predictor
USE_HOPSWORKS=true
FEATURE_GROUP_NAME=aqi_features_clean
FEATURE_GROUP_VERSION=2
HOPSWORKS_UPLOAD_CHUNK_SIZE=3500
HOPSWORKS_UPLOAD_RETRY_COUNT=3
MODEL_NAME=uae_aqi_forecast_model
MODEL_VERSION=1
```

Important learning: .env lines should not be directly typed into PowerShell. In PowerShell, environment variables must be set like:

```
$env:MODEL_NAME="uae_aqi_forecast_model"
$env:MODEL_VERSION="1"
```

20. Commands Used

20.1 Run Feature Pipeline

```
python src/feature_pipeline.py --start-date 2024-06-07 --end-date 2026-06-07
```

20.2 Run Training Pipeline

```
python src/training_pipeline.py
```

20.3 Run Dashboard

```
streamlit run src/app.py
```

20.4 Upload Existing Model Artifact to Hopsworks Resources

```
python upload_model_artifact_hopsworks.py
```

20.5 Check Python Syntax

```
python -m py_compile src/app.py
```

21. Final Dashboard Source Card

The dashboard now includes project infrastructure details such as:

Feature Store: Hopsworks Feature Store

Feature Group: aqi_features_clean v2

Model Artifact: Hopsworks Resources / aqi_model_artifacts

Model File: aqi_model.joblib

Metrics File: metrics.json

Live AQI source: Live Open-Meteo current AQI

Future weather source: Live Open-Meteo Forecast API

Historical model feature timestamp: ...

This makes the dashboard suitable for presentation because it clearly shows that the system is connected to Hopsworks and uses live external data.

22. Final Technical Stack

Layer	Technology
Language	Python
Dashboard	Streamlit
ML	Scikit-learn
Charts	Plotly
Feature Store	Hopsworks Feature Store
Model artifact storage	Local + Hopsworks Resources
Data format	Parquet, JSON, CSV
APIs	Open-Meteo AQI and Weather APIs
Environment config	.env
Code editor	VS Code
Assistance tooling	Codex for code edits

23. What Was Achieved

The final project successfully achieved the majority of the original requirements:

Completed

End-to-end AQI prediction system

Multi-city UAE dashboard

Historical feature backfill

Feature engineering

Hopsworks Feature Store integration

Multi-horizon AQI forecasting

Multiple ML model evaluation

Live AQI integration

Future weather integration

AQI trend dashboard

Pollutant analytics

City comparison

Health recommendations

Model explainability section

Hazard alerts

Downloadable outputs

Model artifact stored in Hopsworks Resources

Professional dashboard UI

Partially Completed

Hopsworks Model Registry

The model registry part was attempted, but the Hopsworks project returned a system dataset issue:

No Models dataset exists in project.

Because the Models dataset is reserved and could not be manually created, the model was successfully uploaded to Hopsworks Resources instead.

24. Problems Faced and How They Were Solved

Problem	Cause	Solution
Only 10 days of data was being fetched	API date parameters/default range issue	Added explicit start and end date arguments
Hopsworks schema validation failed	New features did not exist in old feature group	Created clean feature group <code>aqi_features_clean v2</code>
Primary key null error	<code>location_key</code> had missing values	Cleaned and generated valid location keys
Upload timeout	Too many rows uploaded at once	Added chunked upload
Hopsworks returned None	Feature group creation/read instability	Added safety checks and fallback logic
Model Registry failed	Missing Hopsworks Models system dataset	Uploaded model artifact to Hopsworks Resources
API key permission error	Missing dataset create scope	Created new API key with correct scopes
Dashboard timezone crash	<code>tz-aware</code> vs <code>tz-naive</code> timestamps	Added safe timezone-aware timestamp check
UI cards cut off	Spacing/layout issue	Adjusted premium layout
HTML displayed raw	Streamlit markdown HTML issue	Rebuilt HTML strings safely
Dashboard theme mismatch	Old UI code mixed with new UI	Restored premium yellow theme
Prediction card poor hierarchy	Category text too large	Made predicted AQI number more prominent

25. Limitations

Although the system is functional and strong, there are still some limitations:

AQI values may differ from Google or AQI.in

This happens because different platforms use different data providers, station locations, interpolation methods, and update times.

Model Registry issue remained Hopsworks-side

The formal Model Registry could not be used because the project lacked the required system dataset.

Training can take long

Full model training over 100k+ rows and many features can take several hours.

Live API dependency

If Open-Meteo API is unavailable, the system falls back to saved feature store values.

Forecast uncertainty increases with horizon

Day 2 and Day 3 are naturally harder to predict than Day 1.

26. Future Improvements

The project can be improved further by adding:

GitHub Actions automation

Feature pipeline hourly

Training pipeline daily

Scheduled Hopsworks jobs

Automated feature group updates

Better model registry integration

Resolve Hopsworks Models dataset issue

Register model formally when service is available

Model deployment

Deploy using Hopsworks Deployments or FastAPI

More external APIs

Compare Open-Meteo with AQICN, WAQI, or official UAE air quality sources

Deep learning models

LSTM

Temporal CNN

Transformer-based forecasting

Advanced explainability

SHAP plots

Feature contribution by city

Forecast confidence visualization

User alerts

Email or WhatsApp alerts for unhealthy/hazardous AQI

27. Final Conclusion

This project successfully evolved from a basic AQI prediction idea into a full machine learning product. It includes data ingestion, historical backfill, feature engineering, Hopsworks Feature Store integration, model training, evaluation, live AQI updates, future weather-based inference, and a premium Streamlit dashboard.

The most important achievement is that the project works as a real pipeline rather than a static notebook. It can collect data, store features, train models, generate predictions, and explain results in a user-friendly dashboard.

The Hopsworks Feature Store integration was completed successfully through `aqi_features_clean v2`. The formal Model Registry was blocked by a Hopsworks system dataset issue, but the model artifact and metrics were still uploaded successfully to Hopsworks Resources under:

`Resources/aqi_model_artifacts`

Overall, the final system satisfies the core goals of the AQI Predictor project: an end-to-end prediction system, a scalable feature pipeline, a machine learning training workflow, and an interactive dashboard showing real-time and forecasted AQI data.

Appendix: Final Verification Checklist

- Feature data is available locally in data/feature_store.parquet.
- Feature data was uploaded successfully to Hopsworks Feature Store using aqi_features_clean v2.
- The model artifact exists locally at models/aqi_model.joblib.
- Model metrics are stored locally at models/metrics.json.
- The model artifact, metrics file, and README were uploaded to Hopsworks Resources / aqi_model_artifacts.
- The Streamlit dashboard runs with streamlit run src/app.py.
- The dashboard displays Hopsworks source information, live AQI source, future weather source, and historical feature timestamp.